

Práctica 3: Sistema de Telemetría

Autores

- Jaime Vicente Arroyo
- David Canelada Velasco
- Daniel Zhu
- Inés Olimpia Primo
- Paula Sierra Luque
- Javier Tirado Rios

Índice

- 1. Introducción
 - 1.1 Estructura del repositorio
- 2. Objetivos
- 3. Hipótesis y Preguntas
- 4. Descripción de las métricas
- 5. Descripción de los eventos
 - 5.1 Eventos genéricos
 - 5.2 Eventos específicos por partida
- 6. Cálculo de métricas
- 7. Alineamiento de las métricas con los objetivos e hipótesis
- 8. Análisis y métricas
 - 8.1 Instrucciones de Reproducción
 - 8.2 Análisis de Hipótesis y Preguntas de Investigación
 - 8.3 Comentario general resultado de las métricas
- 9. Instrumentalización

1. Introducción

Se ha desarrollado un sistema de telemetría para la asignatura de Usabilidad y Análisis en Videojuegos. Se ha utilizado en el videojuego Aliestrés, que es un simulador de trabajo, en el que van apareciendo diferentes tipos de distracciones para fastidiar al jugador en su objetivo principal: encestar latas en sus correspondientes contenedores.

1.1 Estructura del repositorio

Este repositorio contiene las siguientes carpetas:

- **Análisis de telemetría.** Donde se incluye el código en python del sistema en sí, junto con archivos JSON, con datos de algunas sesiones.
- **Build.** Se incluye la build del juego Aliestrés.
- **UAJ-P3.** Se incluye el proyecto de Unity (versión 6000.0.6of1) de Aliestrés. En **Assets > P3UAJ_** están las clases en C# creadas para adjuntar el sistema de telemetría.

2. Objetivos

Sobre el videojuego de Aliestrés, se evalúa la capacidad del jugador bajo presión. Para esto, ver cómo el aumento de la dificultad afecta al rendimiento y a la atención hacia las distracciones. Se busca comprender cómo la curva de dificultad ascendente (aumento de la velocidad y cantidad de elementos) afecta negativamente al rendimiento de la tarea principal y empeora la gestión de distracciones.

3. Hipótesis y Preguntas

- Si aumentamos la velocidad y cantidad de las latas generadas, el jugador puede llegar a ignorar tareas secundarias.
- El jugador cometerá más errores si se siente saturado.
- ¿En qué nivel de saturación de latas el jugador ignora las distracciones?
- ¿Qué tipos de distracciones generan mayor dispersión de clics?

4. Descripción de las métricas

- **M1.** Número de latas generadas a lo largo de la partida.
- **M2.** Tasa de aciertos / fallos (rendimiento) por jugador y partida.
- **M3.** Distribución de los clicks del jugador (visualizado con un mapa de calor de clicks).
- **M4.** Cantidad de distracciones de cada tipo.
- **M5.** Porcentaje de distracciones resueltas frente al total de distracciones por jugador y por partida.
- **M6.** Tiempo de reacción ante obstáculos por jugador y por partida.

5. Descripción de los eventos

Para los eventos, hemos definido unos parámetros generales:

- **Timestamp (Int64)** : indica el momento de tiempo en el que se ha lanzado el evento en segundos (tiempo POSIX).
- **ID sesión (string)**: ID único generado mediante una librería de IDs (*shortid*).

Estos parámetros los incluyen como atributos todos los eventos.

5.1 Eventos genéricos

- **Inicio de sesión.** Abrir el juego.
 - Tipo (*eventtype*): “*sessionstart*” (string)
- **Inicio de partida.** Darle al botón de *play*.
 - Tipo (*eventtype*): “*matchstart*” (string)
- **Fin de partida.** Momento en el que aparece la carta de despido, o bien el jugador clica *Salir* en el menú de pausa.
 - Tipo (*eventtype*): “*matchend*” (string)
- **Fin de sesión.** Cerrar el juego.
 - Tipo (*eventtype*): “*sessionend*” (string)

5.2 Eventos específicos por partida

- **Click:** El jugador clica en la partida. Se recoge la posición de cada *click* del jugador para la M3.
 - Tipo: “*mouse_click*” (string)
 - Atributos
 - Posición: coordenadas del ratón al hacer click.
- **Aparece lata:** Se genera una lata en la cinta.
 - Tipo: “*can_appears*” (string)
 - Atributos
 - Color de la lata (Enum { Azul, Rojo, Verde, Rosa })
- **Lata encestada:** Al lanzar la lata, especificamos dónde cae (si ha sido encestada en una caja o fallada (suelo/otro)). Aquí no se incluyen las latas de bebida energética, explicadas a continuación.
 - Tipo: “*can_landed*” (string)
 - Atributos
 - Color lata (Enum { Azul, Rojo, Verde, Rosa })
 - Destino (Enum { Contenedor azul, Contenedor rojo, Basura, Suelo })
- **Coger energética:** El jugador agarra la bebida energética y la lanza. La bebida energética se diferencia del resto de latas porque tiene una mecánica propia. Para resolver la distracción del parpadeo, explicado posteriormente, se utiliza esta bebida energética, simplemente agarrándola mediante clic.
 - Tipo: “*energydrinkused*” (string)
 - Atributos
 - Destino (Enum { Contenedor azul, Contenedor rojo, Basura, Suelo })
- **Aparece distracción:** En la escena ocurren diferentes distracciones con las que el jugador tendrá que lidiar, registramos cuándo surge cada distracción, y de cuál se trata.
 - Tipo: “*distraction_spawned*” (string)
 - Atributos
 - Tipo (Enum { Gato, Mosca, Apagón, Compañeros })

- ID de la distracción (para saber a qué evento “Desaparece distracción” corresponde) (int)
- **Desaparece distracción:** La distracción desaparece automáticamente o el jugador la quita.
 - Tipo: “distraction_despawned” (string)
 - Atributos
 - Tipo (Enum { Gato, Mosca, Apagón, Compañeros })
 - ID de la distracción (para saber a qué evento “Aparece distracción” corresponde) (int)
- **Parpadeo:** Distracción especial que ocurre cuando al jugador le empieza a entrar sueño: la pantalla simula el efecto de cerrar y abrir los ojos. Registramos el momento en el que ocurre esta distracción.
 - Tipo “blink” (string)
 - Atributos
 - Tipo (bool)
- **Perder vida:** El jugador encesta una lata en un contenedor de un color distinto al de la lata que ha lanzado, pierde una vida.
 - Tipo: “life_lost” (string)

6. Cálculo de métricas

A continuación se describe cómo hemos calculado cada métrica:

- M1. Sumando el número de eventos de tipo “can_appears”.
- M2. Usando todos los eventos de tipo “can_landed”, sumamos las latas cuyo color y destino corresponden y lo dividimos por la cantidad de eventos de este tipo.
- M3. Se obtiene una gráfica con el evento de “click” y las posiciones del ratón.
- M4. Usando el evento de “distraction_spawned” y sumando cada tipo.
- M5. Calculamos el porcentaje de éxito al lidiar con las distracciones de una partida. Para ello, se usan eventos de tipo “distraction_spawned” y “distraction_despawned”.
- M6. Calculamos el tiempo medio de reacción del jugador ante los obstáculos de cada partida. Para esto, emparejamos cada evento “distraction_spawned” con su respectivo evento “distraction_despawned”, utilizando el atributo *distractionId*. Calculamos la diferencia entre sus atributos *timestamp* y hallamos la media aritmética de todas las distracciones de la partida.

7. Alineamiento de las métricas con los objetivos e hipótesis

La siguiente tabla muestra la relación entre las hipótesis y preguntas planteadas y las métricas utilizadas:

Hipótesis / Pregunta de Investigación	Métricas asociadas	Justificación
H1. Si aumentamos la velocidad y cantidad de las latas generadas, el jugador puede llegar a ignorar tareas secundarias.	M1, M5	Se cruza la cantidad de latas (M1) con el porcentaje de resolución de distracciones (M5) para comprobar si a mayor carga de trabajo principal, menor atención a lo secundario.
H2. El jugador cometerá más errores si se siente saturado.	M1, M2	Se compara el volumen de latas generadas (M1) con la tasa de aciertos y fallos (M2) para identificar si el rendimiento principal decae en los picos de estrés.
P1. ¿En qué nivel de saturación de latas el jugador ignora las distracciones?	M1, M6	Se utiliza el nivel de saturación (M1) junto con el tiempo de reacción (M6) para encontrar numéricamente el umbral de carga cognitiva máxima del jugador.
P2. ¿Qué tipos de distracciones generan mayor dispersión de clics?	M3, M4	Se analiza el mapa de calor (M3) en función de la cantidad y tipo de distracciones de cada partida (M4) para ver qué eventos provocan un comportamiento más agresivo con el ratón.

8. Análisis y métricas

8.1 Instrucciones de Reproducción

Para garantizar la reproducibilidad y automatización de este análisis, se ha desarrollado un script en Python (análisis_completo.py).

- **Requisitos:** Tener instalado Python 3, la librería Pandas (pip install pandas) y la librería Matplotlib (pip install matplotlib).
- **Ejecución:** En la carpeta ya están los jsons con las sesiones usadas para este análisis, con lo que para reproducirlo solo es necesario ejecutar el comando ***python analisiscompleto.py desde su carpeta (\Análisis de telemetría en la raíz del repositorio).*** En caso de querer usar sesiones nuevas basta con alojar los archivos ***__json*** de telemetría en la misma carpeta que el script (***\Análisis de telemetría***). Los ***.json*** se generan en la dirección ***\AppData\LocalLow\DefaultCompany\AliEstrés***. El sistema procesará automáticamente todas las trazas y mostrará el cálculo de las métricas M1 a M6 por consola sin requerir intervención manual, los heatmaps de las sesiones están guardados en la carpeta ***\Análisis de telemetría\heatmaps***

8.2 Análisis de Hipótesis y Preguntas de Investigación

Hipótesis 1: "Si aumentamos la velocidad y cantidad de las latas generadas, el jugador puede llegar a ignorar tareas secundarias."

Sorprendentemente, los datos refutan parcialmente esta hipótesis. Al observar la Métrica 5 (Porcentaje de distracciones resueltas) en contraste con la Métrica 1 (Latas generadas), vemos que el jugador no abandona las tareas secundarias bajo presión:

- Con 74-75 latas, resuelve entre el 50% y el 57% de las distracciones.
- Con 119 latas, resuelve el 62.50%.
- En el pico máximo de estrés (167 latas), alcanza su tasa máxima de resolución: 66.67%.

Conclusión: El diseño de las distracciones (gato, mosca, luces) es lo suficientemente intrusivo como para obligar al jugador a lidiar con ellas. En lugar de ignorarlas al estar saturado, el jugador entra en un estado de "pánico activo", lo cual se evidencia en el aumento masivo de clics (de 96 a 298), intentando limpiar la pantalla desesperadamente.

Hipótesis 2: "El jugador cometerá más errores si se siente saturado."

Los datos muestran una estabilización del rendimiento que matiza esta hipótesis. Si cruzamos la M1 (Saturación de latas) con la M2 (Tasa de aciertos):

- En la partida de 75 latas, el jugador logró un excepcional 75% de acierto.
 - Sin embargo, al dar el salto a 119 y 167 latas, la precisión cae y se estanca en torno al 65% (65.15% y 65.98% respectivamente).

Conclusión: Existe una penalización inicial en el rendimiento cuando se supera el umbral de las 100 latas, cayendo la precisión un 10%. Sin embargo, una vez alcanzado este nivel de estrés, el jugador parece encontrar un "tope" de error, manteniendo una tasa de acierto del 65% independientemente de si le lanzan 120 o 160 latas.

Pregunta 1: "¿En qué nivel de saturación de latas el jugador ignora las distracciones?"

Basándonos en la M6 (Tiempo medio de reacción) y cruzando con la M1, podemos definir el perfil cognitivo del jugador ante la saturación. En las partidas de intensidad baja y extrema (75 y 167 latas), el jugador tarda casi 10 segundos de media en reaccionar a una distracción. Sin embargo, en la partida media de 119 latas, su tiempo de reacción bajó drásticamente a 4.66 segundos.

Conclusión: El jugador nunca llega a ignorar las distracciones por completo. Más bien, los datos sugieren que en torno a las 110-120 latas el jugador entra en su estado de "flujo" (flow state) óptimo, reaccionando rapidísimo (4.66s). Cuando se rompe ese umbral y se acerca a las 170 latas, la sobrecarga cognitiva es tan grande que, aunque sigue resolviendo las distracciones (M5), su cerebro tarda el doble de tiempo (casi 10 segundos) en procesar visualmente el obstáculo.

Pregunta 2: "¿Qué tipos de distracciones generan mayor dispersión de clics?"

Al cruzar la cantidad y tipo de distracciones (M4) con los mapas de calor (M3), observamos que partidas con alta presencia de las distracciones de "Mosca" y "Apagóon", generan clics mucho más dispersos por toda la pantalla. Por el contrario, cuando las distracciones son mínimas, el mapa de calor se concentra casi exclusivamente en la zona de la cinta y los contenedores

Conclusión: Se demuestra que ciertas distracciones hacen que el jugador tenga menos control sobre sus clics.

8.3 Comentario general resultado de las métricas

- **M1:** El aumento de latas ha sido el principal motivo de la dificultad. El jugador, se estresaba al principio, pero, al darse cuenta de que la cantidad de las latas es cada vez mayor, se iba adaptando a un ritmo.
- **M2:** La tasa de aciertos/fallos nos ha indicado que, ya que es prácticamente imposible acertar todo, el jugador asumía un porcentaje pérdida (sacrificando un tercio de las latas) para no perder el control del resto de la partida.
- **M3:** En el mapa de calor de clics, hemos podido observar cómo actuaba ante el pánico el jugador, ya que nos ha permitido comprobar de un vistazo si la partida estaba bajo control con clicks ordenados y bien distribuidos o si colapsó intentando resolver las distracciones de manera desordenada por toda la pantalla.
- **M4:** La cantidad de distracciones de cada tipo nos ha enseñado que la variedad de distracciones es peor que la cantidad. Sufrir distintos eventos por ejemplo una combinación de gato + apagón + sueño ha roto el bucle de juego mucho más rápido que la repetición de un mismo problema.
- **M5:** Hemos confirmado que los obstáculos logran su objetivo: desconcentrar al jugador, ya que siempre tenía que ocuparse de las distracciones sin importar el estado de las latas.
- **M6:** El tiempo de reacción ha medido en qué estado se encontraba el jugador. Identifica en qué punto el usuario está aburrido, cuándo alcanza su pico de concentración máxima, y cuándo se satura.

9. Instrumentalización

- Clases modificadas:
 - GameManager
 - MenuManager
 - LataManager
 - Cursor
 - TiredEvent
 - EventManager
 - EventoJefe
 - EventoJefe2
 - EventoJefe3
 - Gatete
 - LightController
 - Destructor3000
- Clases auxiliares añadidas:
 - Destructor3000Azul
 - Destructor3000Rojo
 - SueloDetector